

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH MẠNG

**Đề tài: Ironfront Reborn – Trò chơi bắn súng nhiều người chơi
với tầng mạng TCP/UDP tự xây dựng**

Lớp: D23CTPM01-B

Nhóm bài tập lớn: 10

Sinh viên thực hiện: Nguyễn Minh Toàn

Mã sinh viên: B23DCCN832

Báo cáo kiến trúc và giao diện hệ thống

Phần cá nhân: *Phân hệ tầng giao vận UDP (Transport)*

Hà Nội – 2026

Mục lục

I. NỘI DUNG NHÓM	3
1. Mô tả kiến trúc chung của hệ thống	3
1.1. Bài toán và phạm vi	3
1.2. Kiến trúc ba tiến trình	3
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu	5
1.4. Kiến trúc phân lớp mã nguồn	5
1.5. Kiến trúc triển khai	6
1.6. Phân công theo phân hệ	7
2. Mô tả thiết kế chung của hệ thống	7
2.1. Mô hình thẩm quyền	7
2.2. Luồng nghiệp vụ đầu – cuối	8
2.3. Các kỹ thuật netcode cốt lõi	9
2.4. Thiết kế giao thức	9
2.5. Tham số thời gian	10
2.6. Phân loại đối tượng đồng bộ	11
2.7. Bảo mật ở mức phù hợp phạm vi	11
2.8. Các quyết định kiến trúc đã chốt	11
2.9. Quy trình phát triển và bảo đảm chất lượng	11
II. NỘI DUNG CÁ NHÂN	13
3. Biểu đồ và giao diện nội dung cá nhân thực hiện	13
3.1. Kiến trúc thư viện Transport	13
3.2. Định dạng gói tin GSP	13
3.3. Bất tay không lưu trạng thái	14
3.4. Máy trạng thái kết nối	15
3.5. Cơ chế ack bằng bitfield và gửi lại	15
3.6. Kênh, phân mảnh và kiểm soát nghẽn	16
3.7. Giao diện lập trình và giao diện chẩn đoán	17
4. Luồng xử lý và giải pháp đã ứng dụng	17
4.1. Luồng thiết lập kết nối UDP có xác thực	18
4.2. Luồng gửi và nhận một bản tin	19
4.3. Luồng phát hiện mất gói và gửi lại	21
4.4. Bốn kênh truyền và luồng phân loại khi nhận	22
4.5. Luồng cắt và ghép mảnh bản tin lớn	22
4.6. Luồng giữ kết nối, kiểm soát nghẽn và ngắt kết nối	23
4.7. Bộ giả lập mạng, ghi phát lại gói và đo hiệu năng	24
4.8. Kiểm thử phân hệ	24

I. NỘI DUNG NHÓM

1. Mô tả kiến trúc chung của hệ thống

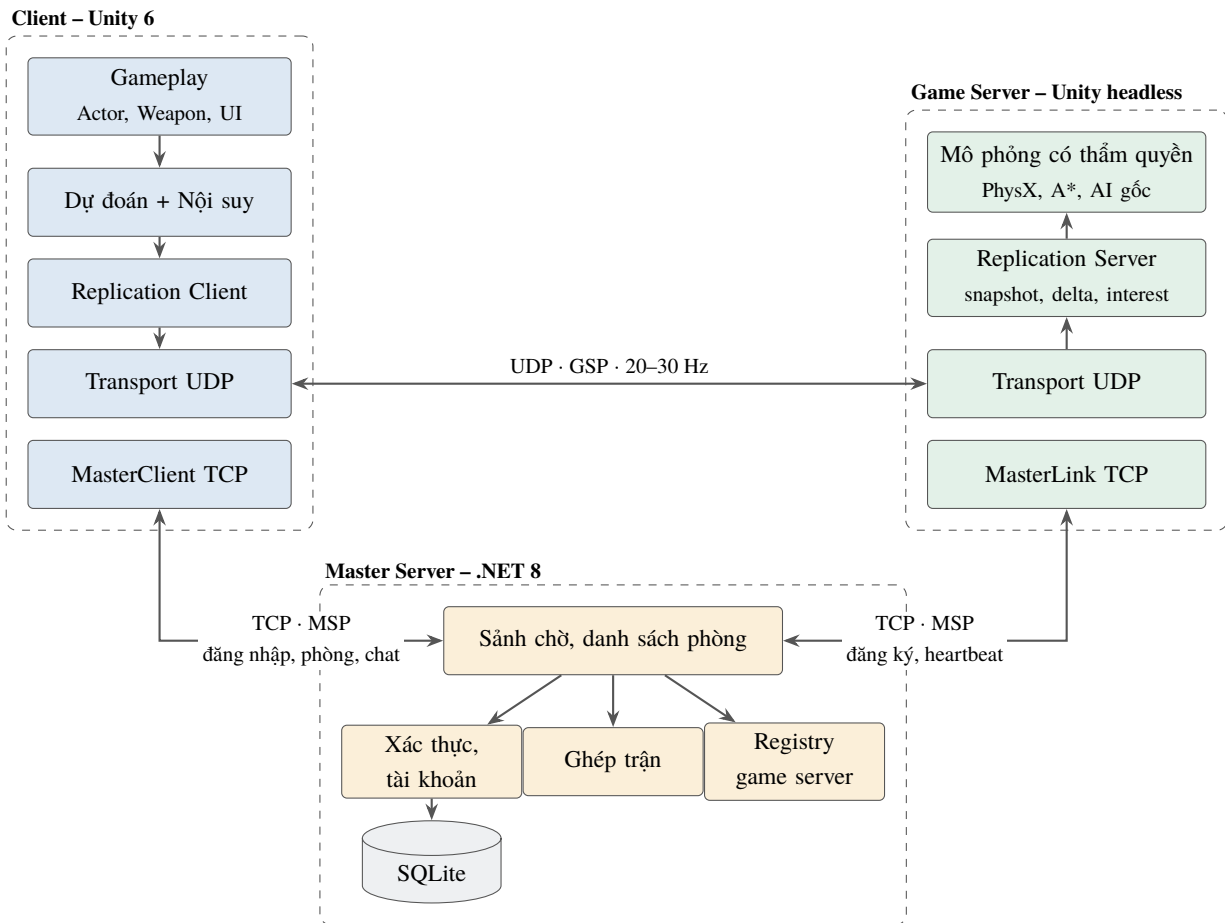
1.1. Bài toán và phạm vi

Ironfront Reborn chuyển mã nguồn trò chơi bắn súng góc nhìn thứ nhất *Ravenfield Beta 5* (Unity 6000.3, vốn chỉ chơi đơn) thành một trò chơi **nhiều người chơi, máy chủ nắm quyền quyết định** (server-authoritative). Yêu cầu cốt lõi của môn Lập trình mạng: **toàn bộ tầng mạng được viết tay trên socket TCP và UDP** thô qua `System.Net.Sockets` — không dùng Mirror, Photon, Netcode for GameObjects, KCP, gRPC hay WebSocket.

Tiêu chí	Giá trị thiết kế
Quy mô một trận	16 người chơi thật + 32 bot AI (tối đa 64 actor, 16 phương tiện)
Tần số mô phỏng / snapshot / gửi input	30 Hz / 20 Hz / 30 Hz
Băng thông xuống mỗi client	≤ 8 KB/s (đo trên server đã triển khai: 1,67 KB/s)
Triển khai	LAN trước, sau đó VPS công cộng (Docker Compose, TLS cho kênh TCP)
Công nghệ	.NET 8 (master server, công cụ), netstandard2.1 (thư viện dùng chung), Unity 6 (client và game server headless)

1.2. Kiến trúc ba tiến trình

Hệ thống gồm **ba loại tiến trình** độc lập, giao tiếp qua **hai giao thức tự định nghĩa**: **MSP** (Master Server Protocol, trên TCP) cho dịch vụ sảnh chờ và **GSP** (Game Server Protocol, trên UDP) cho lưu lượng trong trận. Hai giao thức gặp nhau ở đúng một điểm: **joinTicket** được master ký HMAC và game server tự kiểm tra.



Hình 1.1. Kiến trúc tổng thể ba tiến trình và hai giao thức MSP/GSP

Tiến trình	Nền tảng	Trách nhiệm chính	Giao tiếp
Client	Unity 6, bản build thường	Hiển thị, nhận input, dự đoán chuyển động người chơi cục bộ, nội suy actor ở xa, giao diện sảnh và HUD	TCP tới Master; UDP tới Game Server
Game Server	Unity 6 headless (UNITY_SERVER)	Chạy mô phỏng có thẩm quyền (vật lý, AI, bắn, điểm chiếm), tạo snapshot riêng cho từng client, bù trễ	Nhận UDP từ client; TCP tới Master
Master Server	.NET 8 console, SQLite	Tài khoản, phiên, phòng, ghép trận, chat sảnh, registry game server, cấp joinTicket	Nhận TCP từ client và game server

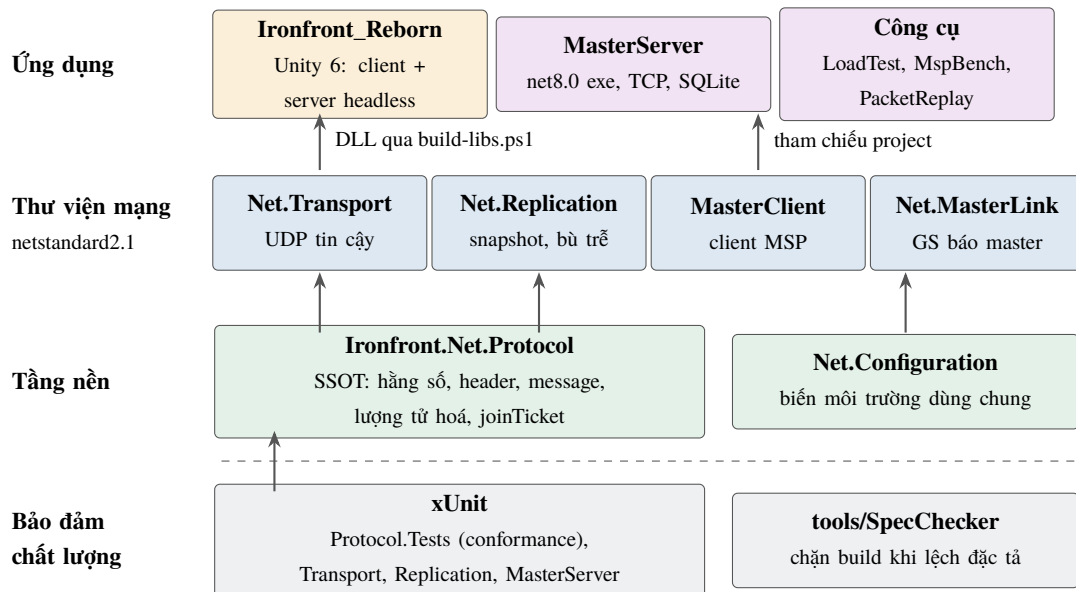
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu

Đặc tính	TCP – Master Server	UDP – Game Server
Dùng cho	Đăng nhập, danh sách phòng, tạo/vào phòng, ghép trận, chat sảnh, heartbeat server	Mọi lưu lượng trong trận: input, snapshot, sự kiện gameplay
Tần suất	Vài gói/phút mỗi client	20–60 gói/giây mỗi chiều
Mất gói	Không chấp nhận (mất LOGIN_RES là treo màn hình)	Chấp nhận với trạng thái: snapshot sau thay thế snapshot trước
Nếu đổi chỗ	Phải tự viết lại độ tin cậy mà TCP đã làm tốt	Head-of-line blocking: một gói mất chặn mọi gói sau; Nagle + delayed ACK thêm 40–200 ms
Đóng khung	Tiền tố độ dài 4 byte + JSON	Datagram tự nhiên, header nhị phân 16 byte

Không dùng WebSocket: WebSocket chạy trên TCP nên thừa hưởng nguyên vẹn head-of-line blocking, cộng thêm chi phí bất tay HTTP — một ràng buộc chỉ cần cho trình duyệt, không cần cho client desktop.

1.4. Kiến trúc phân lớp mã nguồn

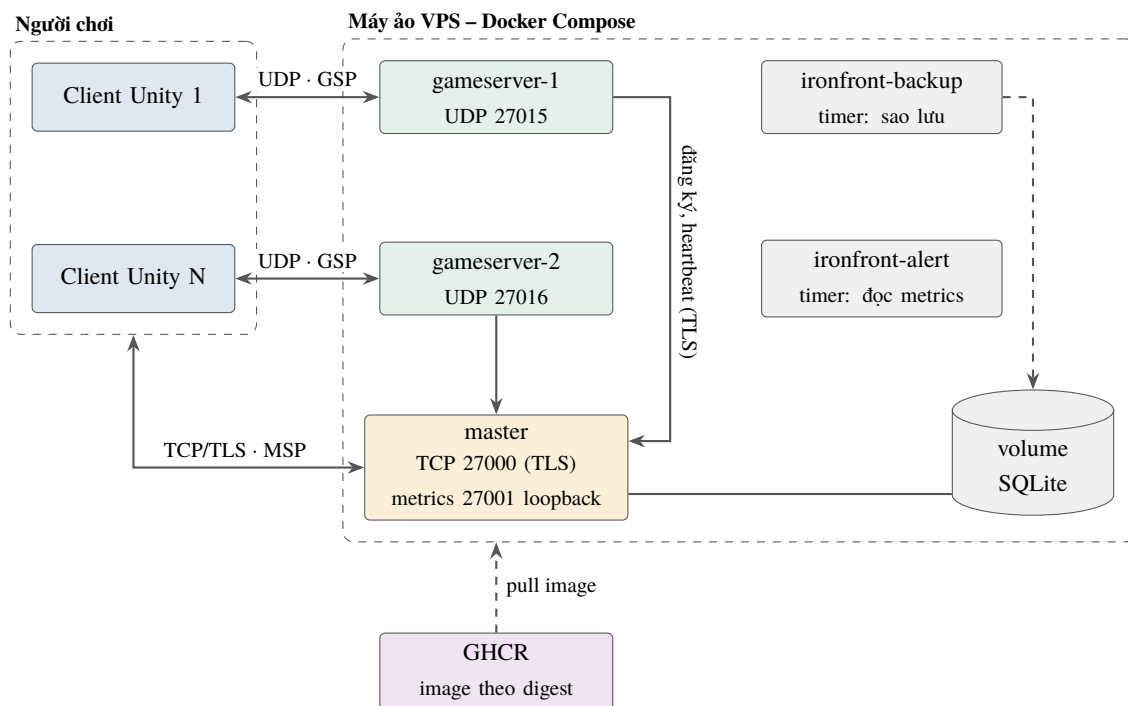
Mã nguồn chia thành các thư viện .NET thuần (netstandard2.1) **không bao giờ tham chiếu** UnityEngine; dự án Unity dùng chúng dưới dạng DLL do tools/build-libs.ps1 sinh vào Assets/Plugins. Nhờ vậy logic mạng được kiểm thử bằng xUnit trong vài giây, không cần mở Unity Editor.



Hình 1.2. Phân lớp thư viện và quan hệ phụ thuộc giữa các project trong solution

Project	Loại	Dòng mã	Vai trò
Ironfront.Net.Protocol	netstandard2.1	5 300	Nguồn sự thật duy nhất của định dạng dây
Ironfront.Net.Transport	netstandard2.1	4 100	Độ tin cậy trên UDP, kênh, phân mảnh, nghẽn, giả lập mạng
Ironfront.Net.Replication	netstandard2.1	18 500	Snapshot, delta, interest, tick server, bù trễ, trận đấu
Ironfront.MasterServer	net8.0 exe	4 300	Framing, dispatch, xác thực, sánh, ghép trận, TLS, metrics
MasterClient, MasterLink	netstandard2.1	1 250	API cho client Unity và liên kết game server – master
Assets/Scripts/Net	Unity C#	14 600	Lớp keo Unity: bootstrap, dự đoán, hiển thị, giao diện

1.5. Kiến trúc triển khai



Hình 1.3. Sơ đồ triển khai trên một máy ảo: một master và hai game server headless

Image được kéo từ GHCR theo digest cố định; bí mật (IRONFRONT_SHARED_SECRET, chứng chỉ TLS) lấy từ tệp .env không commit. Sao lưu và cảnh báo chạy bằng systemd timer ngoài container, vì bộ phát hiện sự cố phải nằm ngoài thứ nó giám sát. Hạ tầng còn có cấu hình Terraform (tạo VM, cloud-init) và fly.io làm phương án thay thế.

1.6. Phân công theo phân hệ

Sinh viên	Mã SV	Phân hệ	Phạm vi mã nguồn chính
Nguyễn Thành Trung	B23DCCN861	Client Unity	Assets/Scripts/Net/Client, Net/Input
Nguyễn Minh Toàn	B23DCCN832	Tầng giao vận UDP	Ironfront.Net.Transport
Nguyễn Duy Nghĩa	B23DCCN600	Replication, mô phỏng server	Ironfront.Net.Replication, Net/Server
Nguyễn Tự Kiên	B23DCCN465	Master Server TCP, hạ tầng	Ironfront.MasterServer, MasterClient, infra

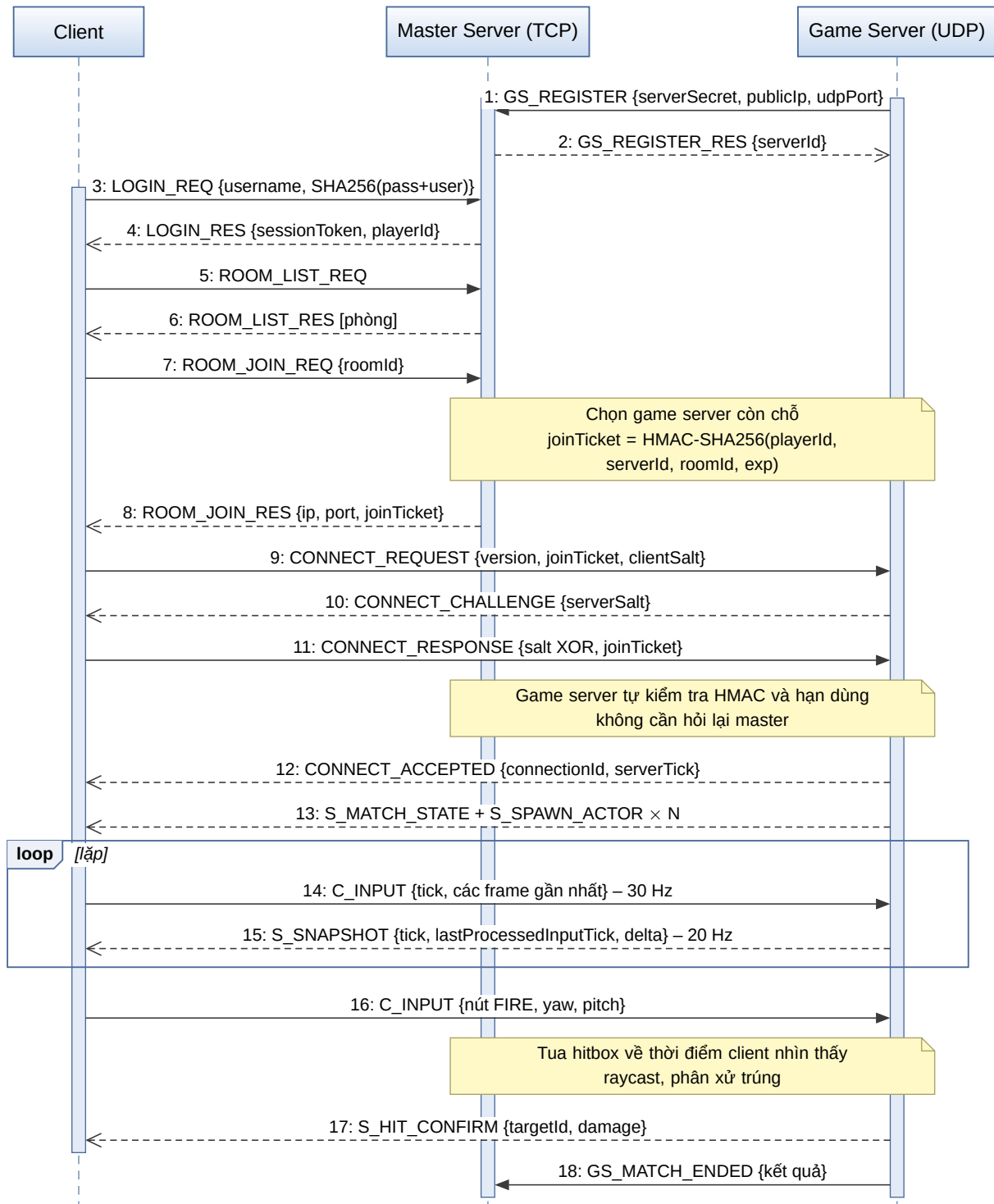
2. Mô tả thiết kế chung của hệ thống

2.1. Mô hình thẩm quyền

Hệ thống **hoàn toàn do server quyết định**: client chỉ gửi *ý định* (input), không bao giờ gửi kết quả. Nhờ vậy gian lận tốc độ, dịch chuyển tức thời, bắn nhanh hay đạn vô hạn trở nên bất khả thi về cấu trúc thay vì chỉ bị phát hiện.

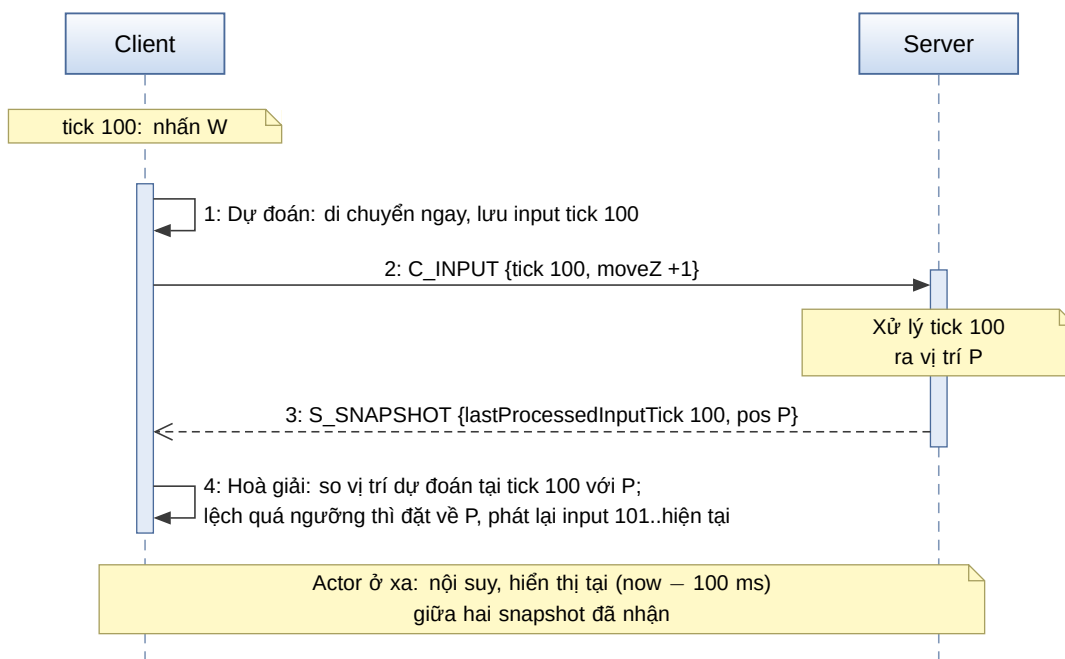
Vấn đề	Ai quyết định	Client được làm gì
Vị trí nhân vật	Server	Dự đoán ngay, sau đó hoà giải với server
Trúng đạn, sát thương	Server	Hiện hiệu ứng dự đoán, chờ S_HIT_CONFIRM
Máu, chết, hồi sinh	Server	Chỉ hiển thị
Độ tản đạn, giật súng	Server	Giật cục bộ cho cảm giác, không ảnh hưởng viên đạn thật
Bot AI, điểm chiếm, điểm số	Server	Nội suy, hiển thị
Lên xuống ghế phương tiện	Server	Gửi yêu cầu C_SEAT_REQUEST
Camera, UI, âm thanh, ragdoll	Client	Toàn quyền, không đồng bộ

2.2. Luồng nghiệp vụ đầu – cuối



Hình 2.1. Luồng đầu – cuối: đăng nhập, vào phòng, kết nối game server, chơi, kết thúc trận

2.3. Các kỹ thuật netcode cốt lõi

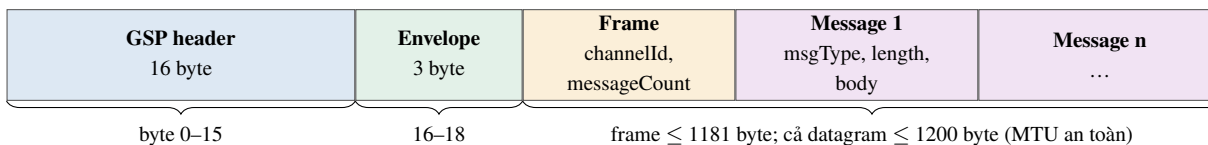


Hình 2.2. Dự đoán phía client, hoà giải với server và nội suy actor ở xa

Kỹ thuật	Mục đích	Tham số
Dự đoán phía client	Người chơi cục bộ di chuyển ngay khi nhấn phím	Cùng MovementSimulation ở client và server
Hoà giải	Sửa sai khi dự đoán lệch, phát lại input chưa xác nhận	Ngưỡng lệch 0,25 m
Nội suy thực thể	Actor ở xa chuyển động mượt dù snapshot chỉ 20 Hz	Bộ đệm 100 ms
Bù trễ	Người ping cao vẫn bắn trúng thứ họ thấy	Tua tối đa 200 ms, lịch sử hitbox 1 s

2.4. Thiết kế giao thức

2.4.1. GSP – cấu trúc một datagram PAYLOAD



Hình 2.3. Ba lớp của một datagram PAYLOAD: header GSP, envelope kênh của transport, frame message của ứng dụng

Kênh	Kiểu	Dữ liệu	Khi mất gói
0	Không tin cậy, không thứ tự	Ping/pong	Bỏ qua
1	Không tin cậy, có thứ tự (server → client)	S_SNAPSHOT, S_WEAPON_FIRE, S_VEHICLE_SNAPSHOT	Bỏ qua; gói cũ bị hủy
2	Tin cậy, có thứ tự	Sự kiện gameplay, spawn/despawn, chat, C_ACK_BASELINE	Gửi lại tối khi được ack
3	Không tin cậy, có thứ tự (client → server)	C_INPUT, C_VEHICLE_INPUT	Bỏ qua; bù bằng dư thừa

Độ tin cậy áp dụng **theo kênh**, không theo kết nối: một sự kiện bị mất trên kênh 2 không làm nghẽn snapshot trên kênh 1 — ưu điểm cốt lõi so với TCP vốn ép mọi thứ vào một luồng.

2.4.2. MSP – khung tin trên TCP

length u32, big-endian	msgType u16, little-endian	body UTF-8 JSON, tối đa 64 KB
----------------------------------	--------------------------------------	---

Hình 2.4. Khung tin MSP: tiền tố độ dài giải quyết bài toán ranh giới thông điệp trên luồng byte TCP

2.4.3. joinTicket – cầu nối TCP và UDP

playerId u32	serverId u16	roomId u16	expiresAtUnixMs u64 (60 s)	displayName 16 byte UTF-8	hmac HMAC-SHA256, 32 byte
32 byte dữ liệu được ký					32 byte chữ ký

Hình 2.5. Cấu trúc joinTicket 64 byte

Master ký vé bằng SHARED_SECRET; game server kiểm tra chữ ký bằng FixedTimeEquals và kiểm hạn dùng. Không cần gọi lại master, nên **master có thể dừng giữa trận mà trận vẫn tiếp tục**.

2.5. Tham số thời gian

Tham số	Giá trị	Ghi chú
Tick mô phỏng server	30 Hz (33,3 ms)	Theo đồng hồ thực, không theo số lần FixedUpdate
Snapshot	20 Hz (50 ms)	Rẻ hơn 30 Hz, nội suy bù độ mượt
Gửi input	30 Hz	Mỗi gói lặp lại các frame gần nhất
Bộ đệm nội suy	100 ms	Chịu được mất một snapshot
Cửa sổ bù trễ / lịch sử hitbox	200 ms / 1 s	Chặn lạm dụng ping giả
Keep-alive / timeout UDP	1 s / 10 s	
Heartbeat MSP client / game server	15 s	Phát hiện kết nối TCP nửa mở

2.6. Phân loại đối tượng đồng bộ

Loại	Ví dụ	Cách xử lý
Actor được đồng bộ	Người chơi, bot	Có actorId u16, có trong snapshot, mã hoá delta
Phương tiện	Xe, trực thăng, tàu	Luồng riêng S_VEHICLE_SNAPSHOT
Chỉ ở server	Lịch sử hitbox, dữ liệu AI	Không bao giờ gửi
Chỉ ở client	Camera, UI, particle, ragdoll, âm thanh	Không bao giờ gửi
Sự kiện một lần	Chết, nổ, phát súng, chiếm điểm	Kênh tin cậy có thứ tự
Tĩnh	Địa hình, công trình, điểm hồi sinh	Có sẵn trong scene hai phía

2.7. Bảo mật ở mức phù hợp phạm vi

Mối đe dọa	Biện pháp	Tầng
Tăng tốc, dịch chuyển, bắn nhanh, đạn vô hạn	Server chỉ nhận input; kẹp tốc độ; tự quản lý cooldown và đạn	Replication
Mạo danh người chơi	connectionId gắn với playerId trong joinTicket đã ký	Transport
Khuếch đại giả mạo IP	Bắt tay challenge không lưu trạng thái (cookie HMAC)	Transport
Gói rác, lụt gói	Kiểm protocolId; giới hạn tốc độ theo IP; giới hạn nhóm phân mảnh	Transport
Nghe lén mật khẩu	TLS và ghim vân tay chứng chỉ; SHA-256 phía client, bcrypt phía server	Master
Tràn bộ nhớ, Slowloris	Khung tối đa 64 KB; hạn 30 s trước xác thực; 5 kết nối/IP	Master
SQL injection, dò mật khẩu	Truy vấn tham số hoá; 5 lần/phút/IP; khoá 15 phút sau 10 lần sai	Master

2.8. Các quyết định kiến trúc đã chốt

Mã	Quyết định	Lý do
AD-1	Server nắm quyền, không có listen-server	Thẩm quyền rõ ràng; tránh sửa 21 singleton
AD-2	Game server là Unity headless	Tái dùng PhysX, A*, AI; tiết kiệm 8–12 tuần
AD-3	Không theo đuổi tất định (lockstep)	Random rải rác 27 tệp; PhysX không tất định
AD-4	Ragdoll chỉ là hiệu ứng cục bộ	Tiết kiệm khoảng 1,7 MB/s
AD-5	Transport, Replication là thư viện .NET thuần	Kiểm thử bằng xUnit, tránh xung đột Unity
AD-7	Snapshot không tin cậy, sự kiện tin cậy	Đúng bản chất trạng thái và sự kiện
AD-8	Không dùng WebSocket	Yêu cầu môn học và head-of-line blocking

2.9. Quy trình phát triển và bảo đảm chất lượng

- **Đặc tả giao thức đóng băng** và `ProtocolConstants.cs` là nguồn sự thật duy nhất; `tools/SpecChecker` làm hỏng build nếu hai bên lệch nhau.
- **Tách người viết và người kiểm:** phân hệ Transport viết bit-packing, phân hệ Replication viết bộ test conformance bằng hex viết tay theo đặc tả.
- **CI** chạy trên Ubuntu và Windows: build coi cảnh báo là lỗi, chạy test, kiểm tra lệch đặc tả.

- **Definition of Done:** đã chạy và xem đầu ra, test phân hệ xanh, toàn bộ test xanh, Unity biên dịch sạch, đã merge và viết báo cáo phase.

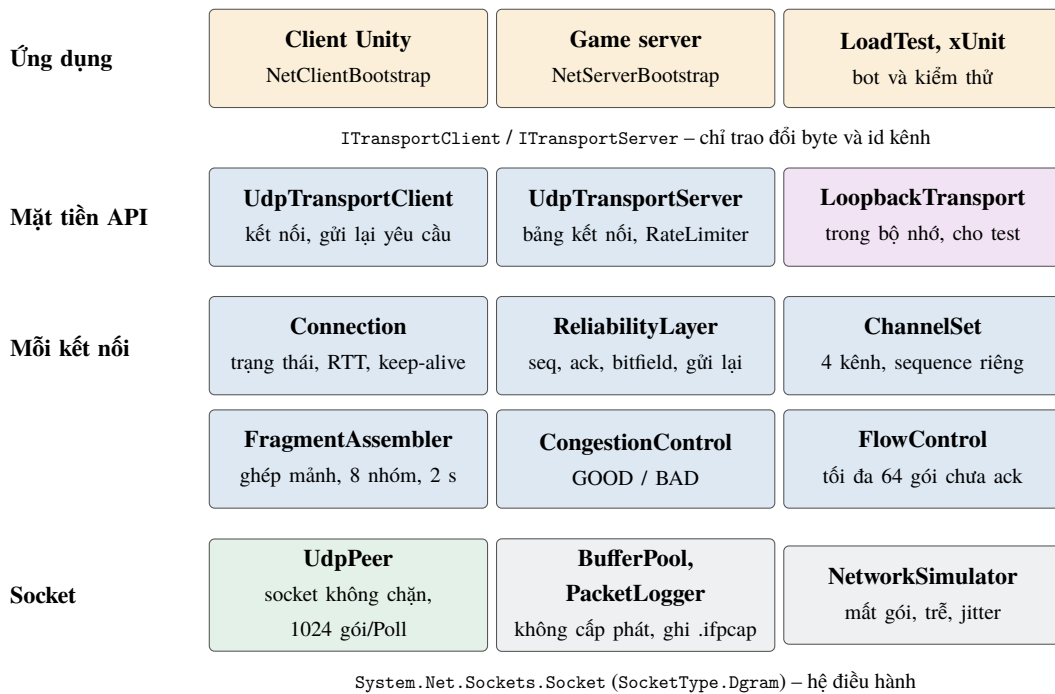
II. NỘI DUNG CÁ NHÂN

Phân hệ tầng giao vận UDP (Transport).

3. Biểu đồ và giao diện nội dung cá nhân thực hiện

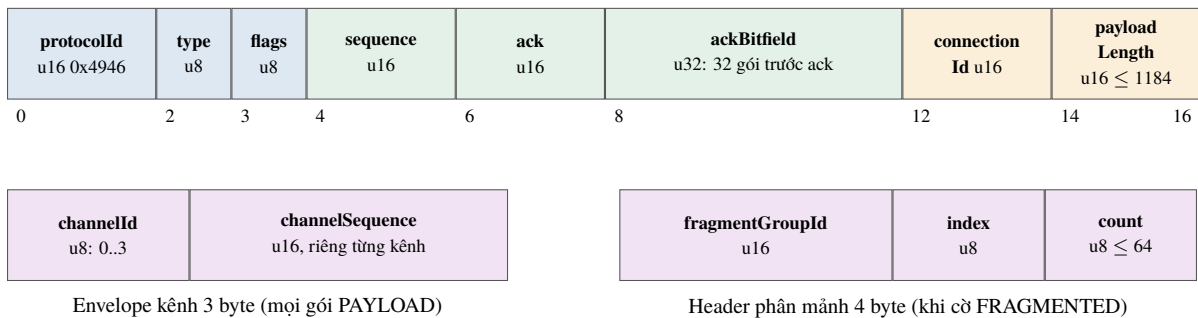
Phân hệ Transport xây dựng **tầng giao vận tin cậy trên UDP từ con số không**, chỉ với Socket kiểu Dgram: đánh số thứ tự, ack bằng bitfield, gửi lại, 4 kênh độc lập, phân mảnh, ước lượng RTT, kiểm soát nghẽn, chống giả mạo và bộ giả lập mạng. Tầng này chỉ biết mảng byte — không biết nội dung là snapshot hay chat — và không tham chiếu UnityEngine.

3.1. Kiến trúc thư viện Transport



Hình 3.1. Biểu đồ thành phần thư viện Ironfront.Net.Transport

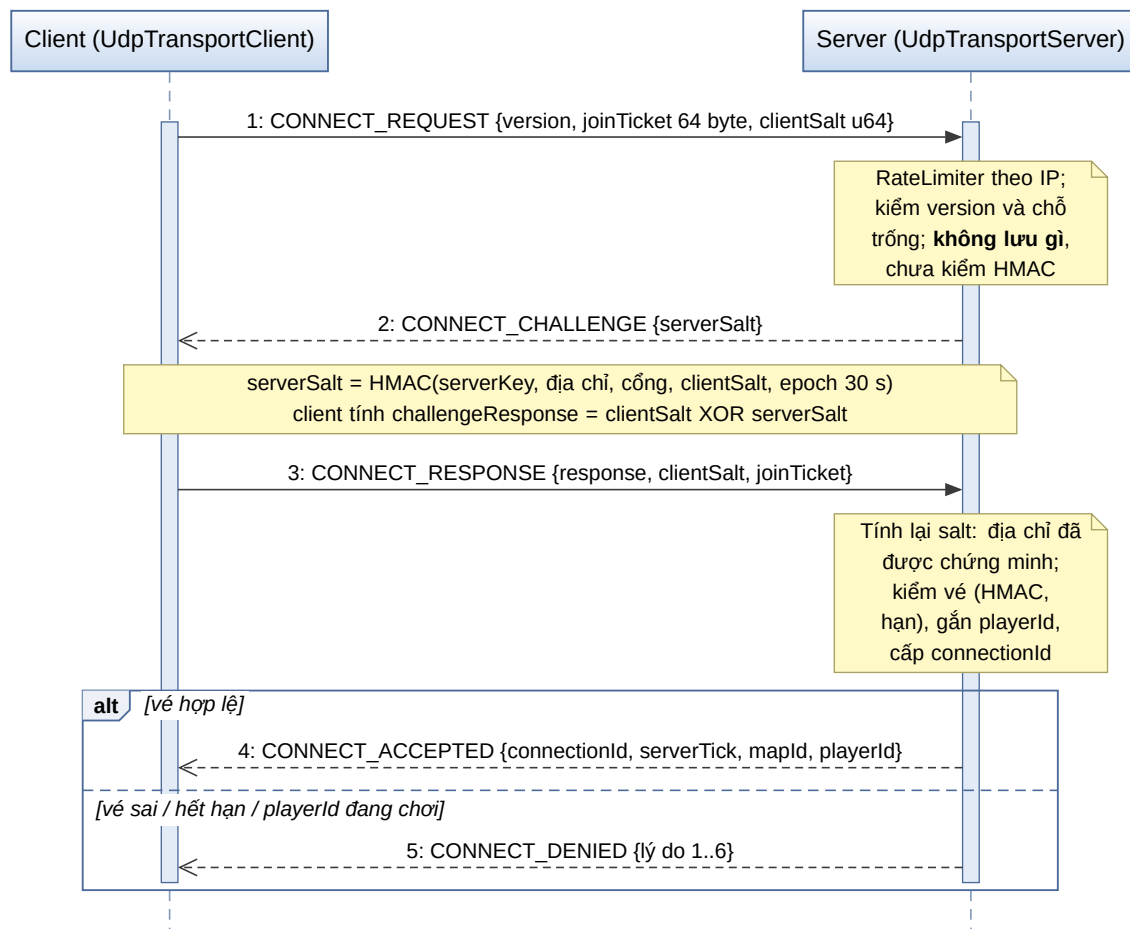
3.2. Định dạng gói tin GSP



Hình 3.2. Header GSP 16 byte (little-endian), envelope kênh và header phân mảnh

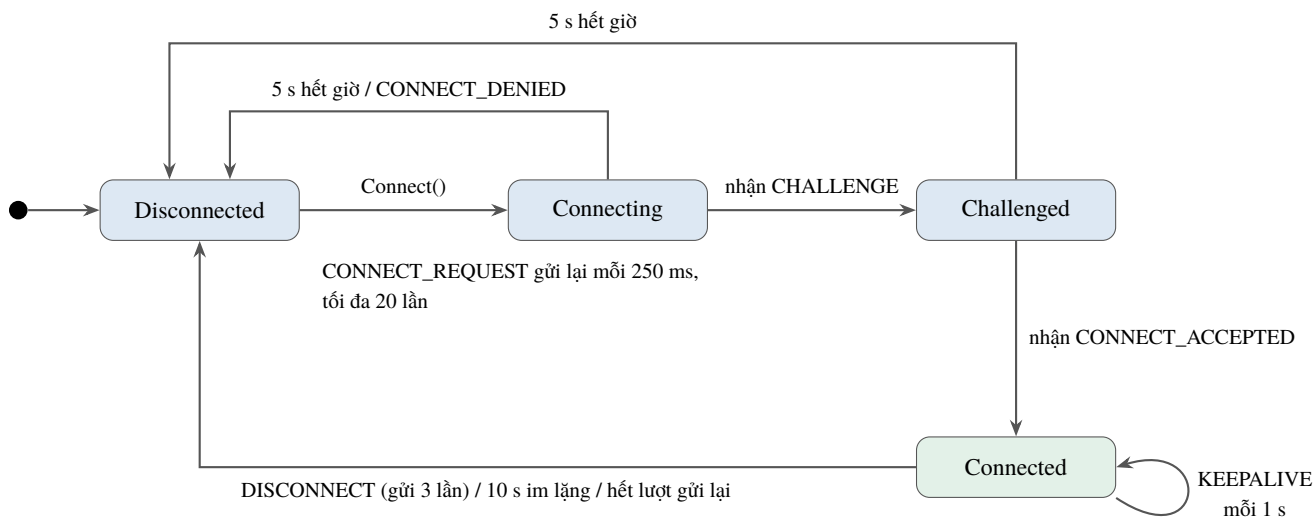
Trường	Giá trị
Bit flags	0 RELIABLE · 1 FRAGMENTED · 2 ORDERED · 3 COMPRESSED (dự phòng) · 4–7 phải bằng 0
packetType	0x01 CONNECT_REQUEST · 0x02 CONNECT_CHALLENGE · 0x03 CONNECT_RESPONSE · 0x04 CONNECT_ACCEPTED · 0x05 CONNECT_DENIED · 0x06 DISCONNECT · 0x07 KEEPALIVE · 0x10 PAYLOAD · 0x11 FRAGMENT

3.3. Bắt tay không lưu trạng thái



Hình 3.3. Bắt tay challenge – response dùng cookie HMAC: server không cấp tài nguyên cho địa chỉ chưa được chứng minh

3.4. Máy trạng thái kết nối

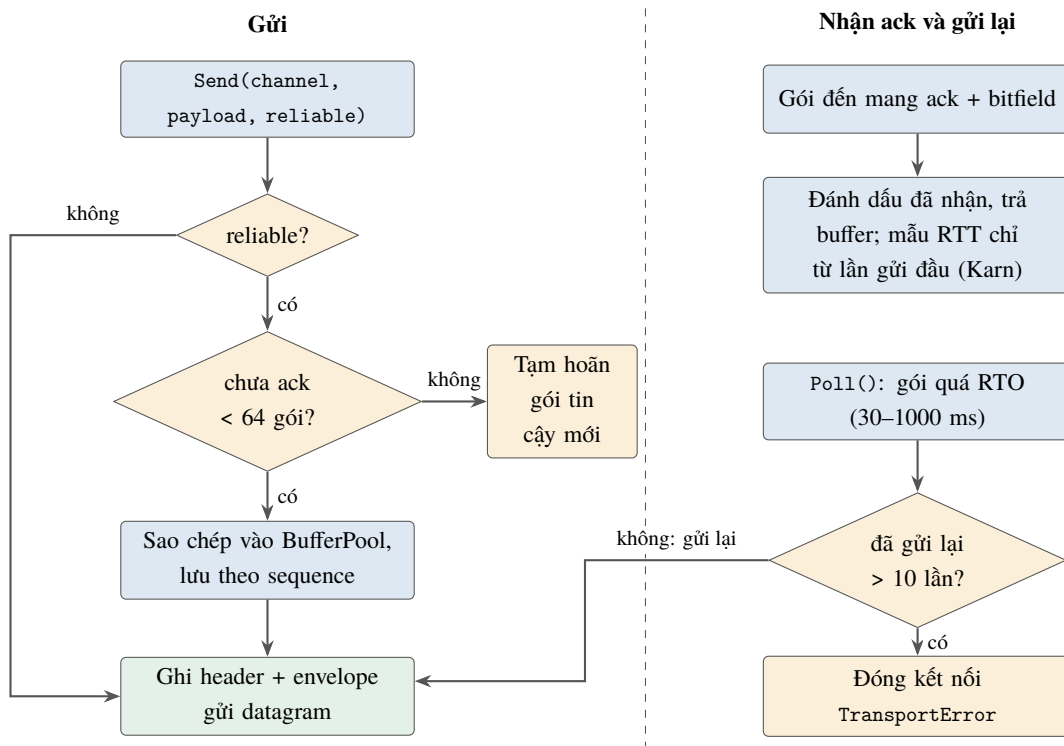


Hình 3.4. Máy trạng thái ConnectionState của một kết nối

3.5. Cơ chế ack bằng bitfield và gửi lại

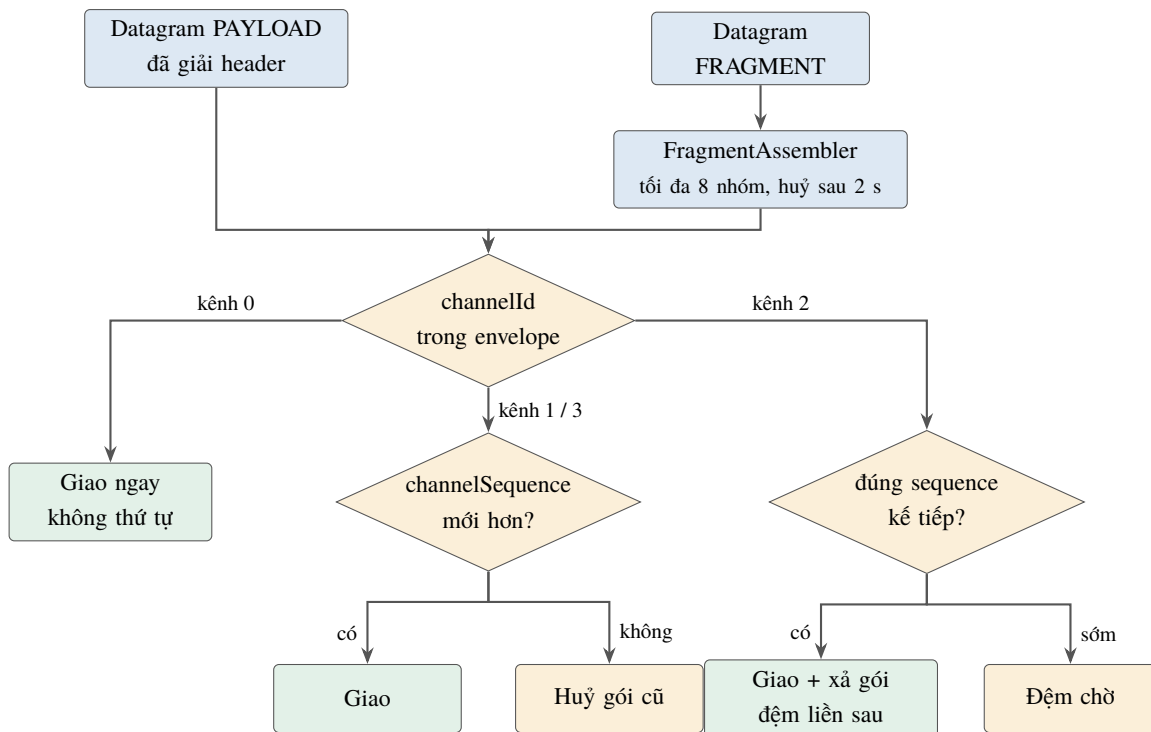
seq 103 đã nhận	seq 102 MẤT	seq 101 đã nhận	seq 100 MẤT	seq 99 đã nhận	seq 98 đã nhận	seq 97 đã nhận
ack = 103	bit0 = 0	bit1 = 1	bit2 = 0	bit3 = 1	bit4 = 1	bit5 = 1

Hình 3.5. Ví dụ ack bitfield: ack = 103, ackBitfield = ...111010; bên gửi biết ngay gói 102 và 100 bị mất. Mỗi gói mang 33 thông tin ack nên không cần gói ACK riêng

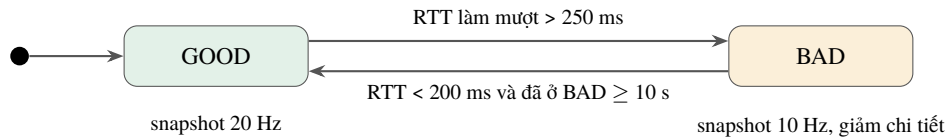


Hình 3.6. Luồng gửi tin cậy, xử lý ack và gửi lại trong ReliabilityLayer

3.6. Kênh, phân mảnh và kiểm soát nghẽn

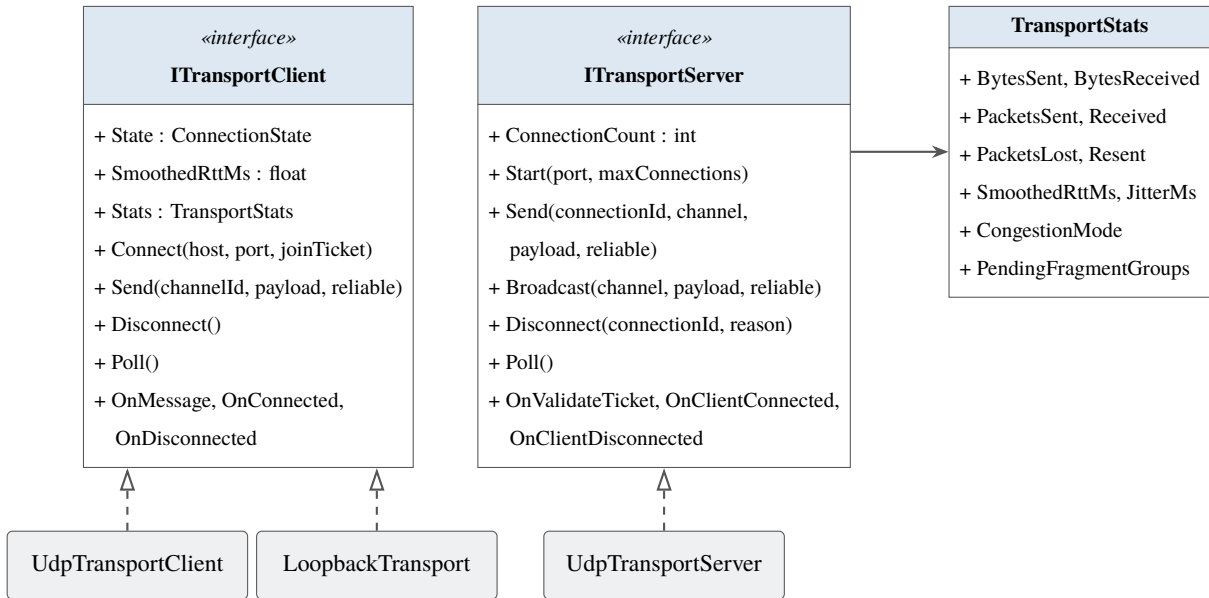


Hình 3.7. Xử lý phía nhận theo kênh: kênh 2 chờ đúng thứ tự, kênh 1 và 3 bỏ gói cũ

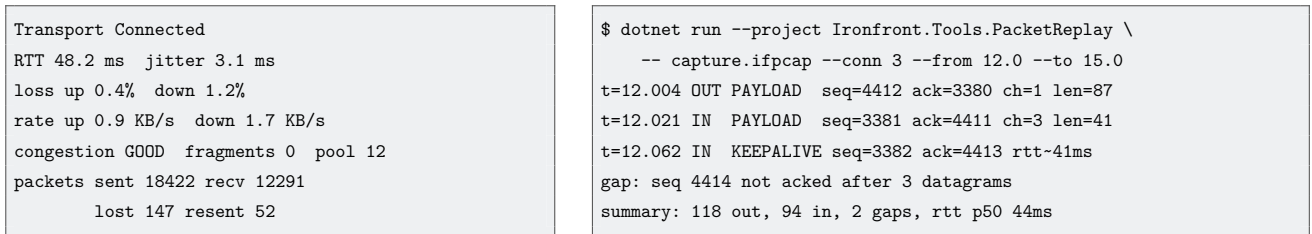


Hình 3.8. Kiểm soát nghẽn hai chế độ có trễ (hysteresis) 250/200 ms; RTT làm vượt theo EWMA 0,9/0,1

3.7. Giao diện lập trình và giao diện chẩn đoán



Hình 3.9. Giao diện lập trình công khai của Transport (đồng bộ từ tuần 1)



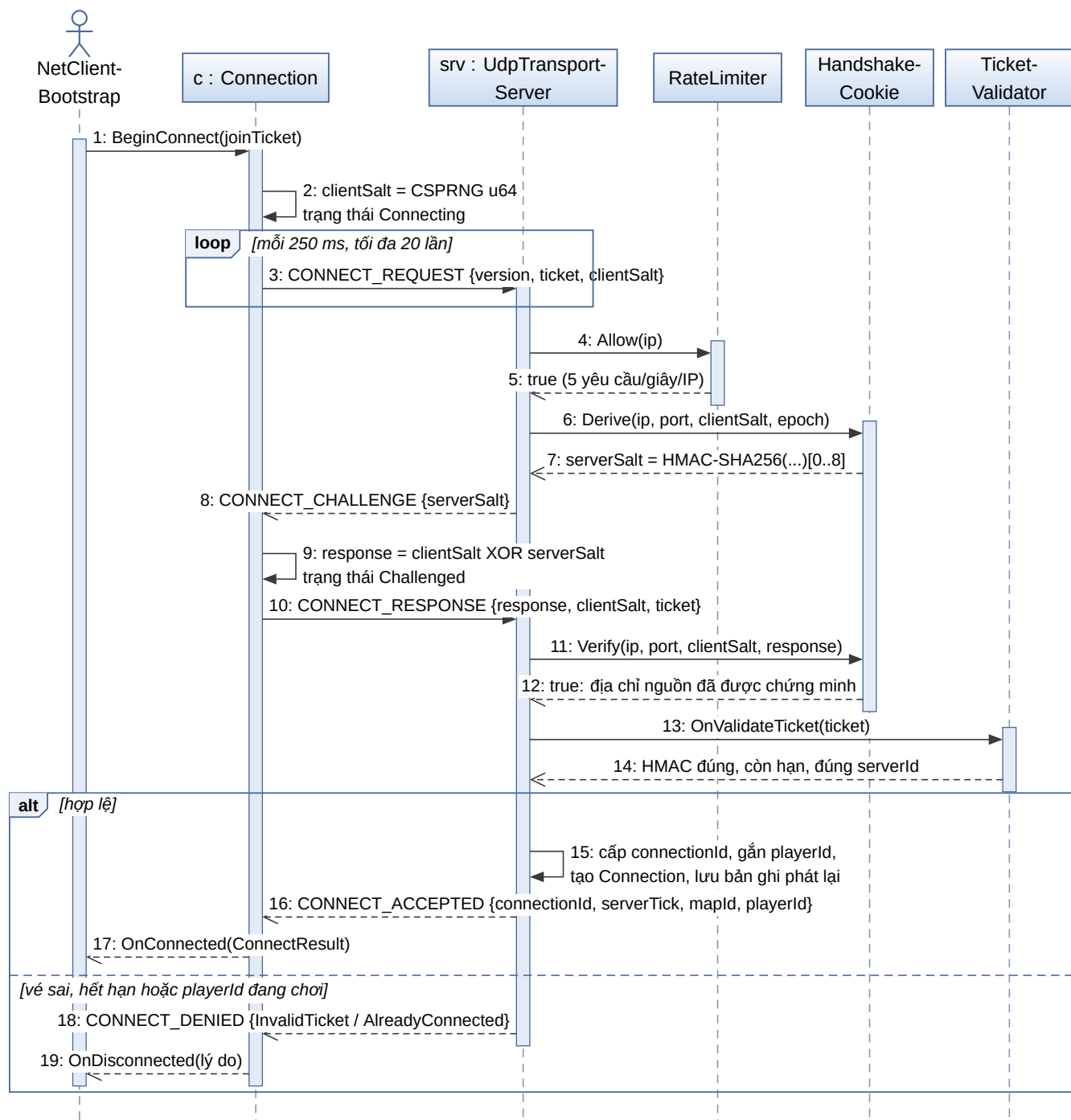
Hình 3.10. Giao diện chẩn đoán: văn bản overlay do TransportDiagnosticsFormatter sinh (trái) và minh họa đầu ra công cụ phát lại gói (phải); các số là minh họa bổ cục

4. Luồng xử lý và giải pháp đã ứng dụng

Mục này đi theo từng đầu việc của phân hệ trong bản mô tả đề tài: thiết lập kết nối UDP, header 16 byte, bắt tay có cookie, ack bằng bitfield và gửi lại, bốn kênh truyền, cắt ghép mảnh, kiểm soát nghẽn, giữ kết nối, bộ giả lập mạng và công cụ đo. Với mỗi đầu việc, báo cáo trình bày **luồng xử lý** (sơ đồ tuần tự theo đúng tên lớp trong mã nguồn), **vấn đề gặp phải** và **giải pháp đã áp dụng**. Các tham số lấy trực tiếp từ mã nguồn Ironfront.Net.Transport và ProtocolConstants.

4.1. Luồng thiết lập kết nối UDP có xác thực

Bài toán. UDP không có kết nối: bất kỳ ai cũng gửi được datagram tới cổng 27015 và điền địa chỉ nguồn giả. Nếu server cấp bộ nhớ, sinh số ngẫu nhiên hoặc kiểm chữ ký ngay khi nhận CONNECT_REQUEST, kẻ tấn công chỉ cần gửi hàng loạt gói với địa chỉ nguồn giả là làm đầy bảng chờ và đẩy người chơi thật ra ngoài. Server cũng phải biết người kết nối chính là người đã được Master Server cấp vé.



Hình 4.1. Biểu đồ tuần tự thiết lập kết nối: server không lưu trạng thái cho tới khi địa chỉ nguồn được chứng minh

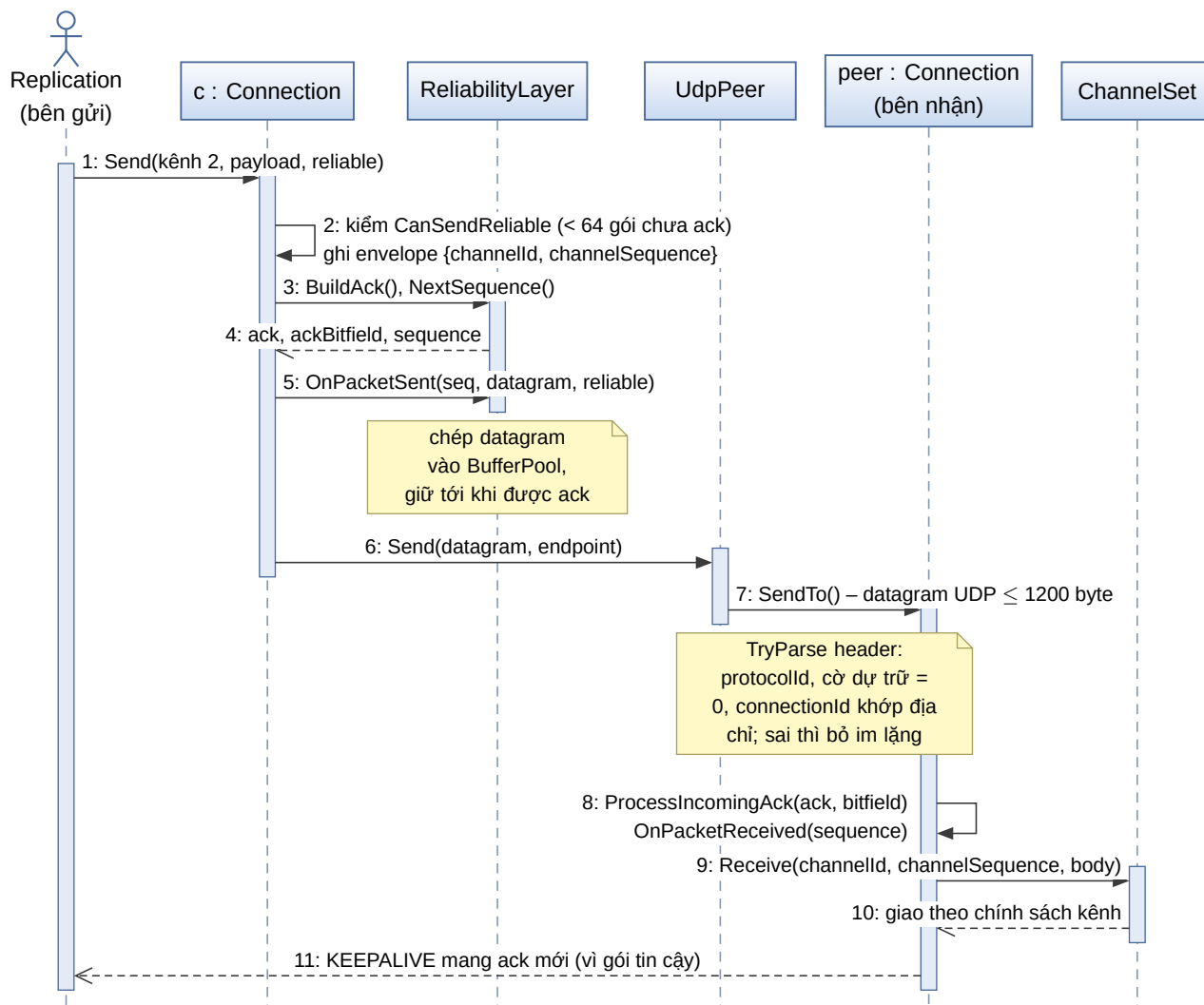
Diễn giải luồng.

1. Client tạo `clientSalt` ngẫu nhiên bằng `RandomNumberGenerator` và gửi `CONNECT_REQUEST` (kích thước cố định, sai kích thước bị bỏ im lặng). Gói được gửi lại mỗi 250 ms, tối đa 20 lần (5 giây), vì chính gói này cũng có thể mất.
2. Server kiểm tra theo thứ tự rẽ trước: giới hạn tốc độ theo IP, phiên bản giao thức, địa chỉ đã có kết nối chưa, còn chỗ trống không. Sau đó nó **không lưu gì** mà tính `serverSalt` bằng HMAC-SHA256 trên (địa chỉ, cổng, `clientSalt`, khung thời gian 30 s) và trả `CONNECT_CHALLENGE` về đúng địa chỉ đã khai.
3. Chỉ client thật sự sở hữu địa chỉ đó mới nhận được challenge và tính được `clientSalt XOR serverSalt`. Khi `CONNECT_RESPONSE` tới, server tính lại cookie (chấp nhận khung hiện tại và khung liền trước) để xác minh — tới đây địa chỉ nguồn mới đáng tin.
4. Lúc này server mới làm việc tốn kém: kiểm HMAC của `joinTicket`, đọc `playerId` và từ chối nếu `playerId` đó đang có kết nối khác. Hợp lệ thì lấy một `connectionId` từ hàng đợi id trống, tạo `Connection` và trả `CONNECT_ACCEPTED`.
5. `CONNECT_ACCEPTED` có thể mất. Server giữ một bản ghi nhỏ theo địa chỉ; nếu client gửi lại `CONNECT_RESPONSE` với đúng đáp án, server gửi lại `CONNECT_ACCEPTED` cũ thay vì cấp kết nối thứ hai.

Vấn đề	Giải pháp đã áp dụng	Mã nguồn
Lụt <code>CONNECT_REQUEST</code> với địa chỉ nguồn giả làm đầy bảng chờ	Cookie không trạng thái kiểu SYN-cookie: <code>serverSalt</code> là HMAC của địa chỉ, cổng, <code>clientSalt</code> và epoch 30 s, tính lại khi cần, không bao giờ lưu	<code>HandshakeCookie.Derive / Verify</code>
Server bị dùng làm công cụ khuếch đại	Challenge chỉ gửi một gói nhỏ về địa chỉ đã khai; giới hạn 5 yêu cầu/giây/IP, bảng giới hạn tối đa 10 000 IP và tự dọn sau 10 s	<code>RateLimiter</code>
Một vé bị lộ mở ra nhiều người chơi	Gắn <code>playerId</code> trong vé với <code>connectionId</code> ; vé thứ hai bị từ chối <code>AlreadyConnected</code> , không đá người đang chơi	<code>_playerIdToConnection</code>
Mất <code>CONNECT_ACCEPTED</code>	Bản ghi phát lại theo địa chỉ, hết hạn sau 10 s	<code>AcceptedHandshake</code>
Con trỏ ack bị gieo sai từ bộ đếm gói điều khiển dùng chung	Gói bắt tay không cập nhật con trỏ ack; gói dữ liệu đầu tiên mới khởi tạo nó	<code>Connection.HandleClientHandshake</code>

4.2. Luồng gửi và nhận một bản tin

Tầng trên (Replication) chỉ gọi `Send(channelId, payload, reliable)` và nhận lại `OnMessage(payload)`. Mọi việc ở giữa — đánh số, dựng header, ghi nhận để gửi lại, đưa vào socket, phân loại theo kênh — là của Transport. Toàn bộ chạy trên luồng gọi `Poll()`, không tạo luồng riêng, nên không cần khoá.

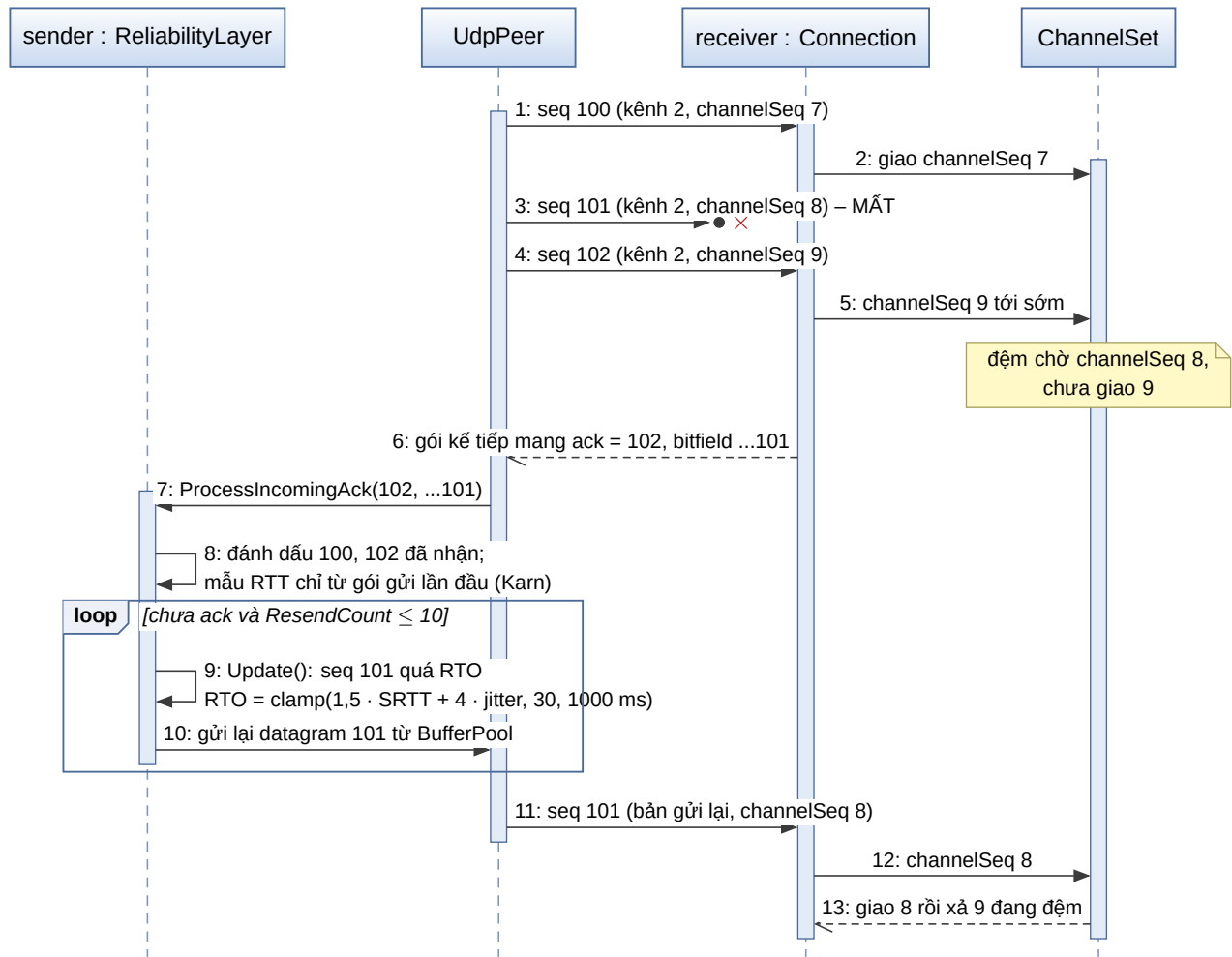


Hình 4.2. Biểu đồ tuần tự gửi – nhận một bản tin tin cậy qua kênh 2

Các giải pháp trên đường nóng.

- **Ack công trên mọi gói (piggyback).** Header nào cũng mang ack và ackBitfield 32 bit, nên một gói xác nhận được 33 gói cùng lúc và không cần gói ACK riêng. Riêng khi nhận gói tin cậy, bên nhận gửi ngay một KEEPALIVE để bên gửi không phải chờ tới nhịp giữ kết nối 1 giây.
- **Không cấp phát trên đường nóng.** Mọi mảng byte lấy từ BufferPool kích thước cố định 1200 byte. Đo trên .NET 8 Release: BufferPool 10,4 ns/lần, ArrayPool.Shared 17,4 ns, new byte[] 16,7 ns; sau khởi động, Poll và Send cấp phát 0 byte.
- **Nhận có giới hạn.** Mỗi lần Poll đọc tối đa 1024 gói; phần còn lại để lượt sau (33 ms). Không giới hạn thì khi bị lụt gói, vòng mô phỏng nằm trọn trong vòng nhận và server ngừng mô phỏng.
- **Bộ đệm socket 1 MB và cảnh báo khi hệ điều hành cắt bớt.** Linux mặc định giới hạn rmem_max khoảng 208 KB mà không báo lỗi; UdpPeer đọc lại kích thước thực tế và ghi cảnh báo. Trên Windows, tắt SIO_UDP_CONNRESET để gói ICMP “port unreachable” không làm hỏng lần nhận sau.
- **Gói lỗi không được trả lời.** Header sai, connectionId không khớp địa chỉ hoặc cờ dự trữ khác 0 đều bị bỏ im lặng — trả lời sẽ biến server thành công cụ dò cống.

4.3. Luồng phát hiện mất gói và gửi lại



Hình 4.3. Biểu đồ tuần tự mất gói trên kênh tin cậy: bitfield chỉ ra gói mất, RTO kích hoạt gửi lại, kênh giữ đúng thứ tự

Vấn đề	Giải pháp đã áp dụng
Biết gói nào mất mà không tốn gói ACK riêng	Cửa sổ nhận 32 bit dịch theo khoảng cách sequence; bên gửi quét ack và 32 bit để xác nhận từng gói (ProcessIncomingAck)
Số thứ tự 16 bit quay vòng sau khoảng 36 phút ở 30 gói/giây	So sánh bằng SequenceMath.IsNewer / Distance theo số học modulo 2^{16} , không dùng phép so sánh thường; có test biên tại 65535 → 0
RTT bị đo sai khi gói đã gửi lại	Thuật toán Karn: chỉ lấy mẫu RTT khi gói được ack mà chưa từng gửi lại; SRTT làm mượt EWMA 0,9/0,1, jitter theo hệ số 0,25
Chờ gửi lại quá lâu hoặc quá dồn	RTO động = $1,5 \times \text{SRTT} + 4 \times \text{jitter}$, kẹp trong [30; 1000] ms
Gói tin cậy mất vĩnh viễn làm kênh có thứ tự treo mãi	Quá 10 lần gửi lại hoặc trùng ô vòng đệm 1024 thì đóng kết nối với TransportError thay vì tiếp tục im lặng (kết nối trông khoẻ nhưng không giao gì)
Bên nhận quá tải	Điều khiển luồng: tối đa 64 gói tin cậy chưa ack; KEEPALIVE quảng bá số gói chờ và mức áp lực, bên gửi tạm dừng gói tin cậy mới khi áp lực > 80%

4.4. Bốn kênh truyền và luồng phân loại khi nhận

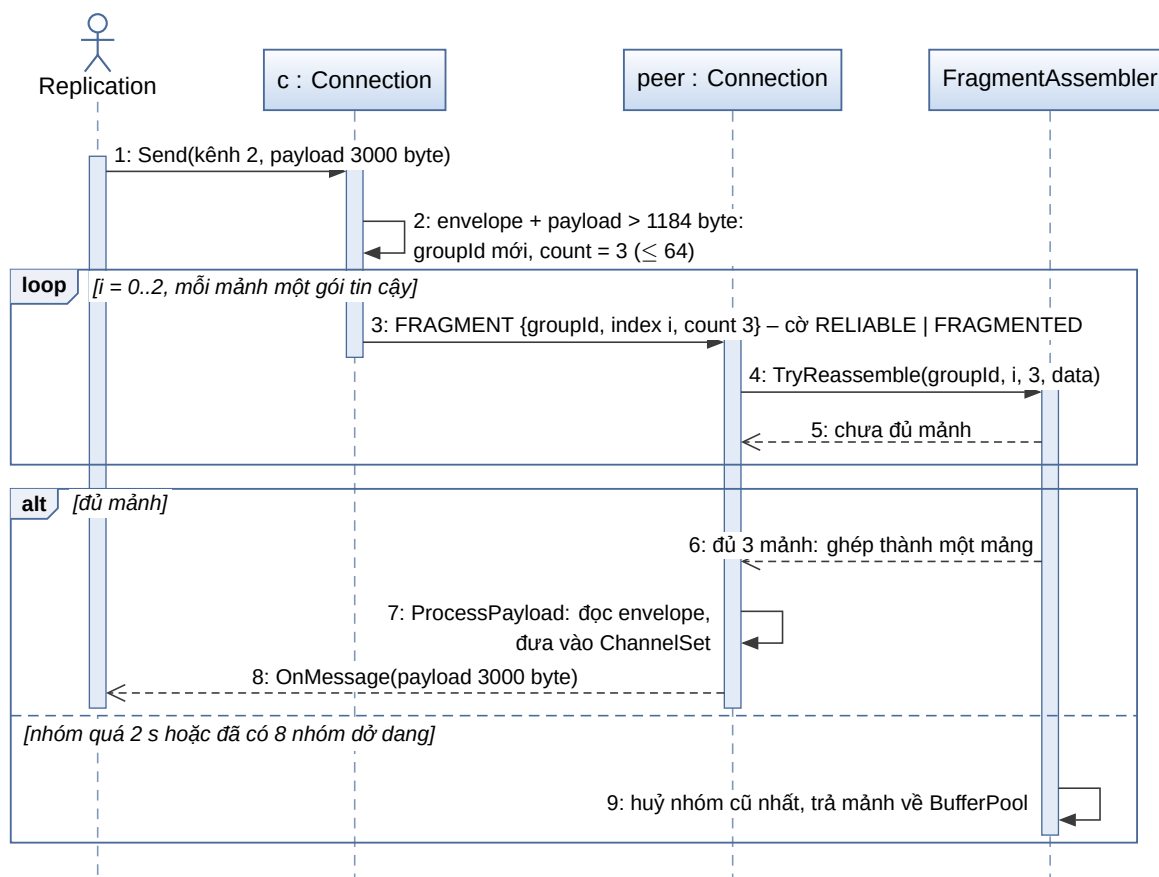
Mỗi gói PAYLOAD có envelope 3 byte gồm channelId và channelSequence **riêng từng kênh**, tách khỏi sequence của kết nối. Nhờ vậy KEEPALIVE hay gói của kênh khác không làm một snapshot trông “cũ”.

Kênh	Đảm bảo	Cách xử lý phía nhận (ChannelSet.Receive)	Dữ liệu
0	Không tin cậy, không thứ tự	Giao ngay	Ping/pong
1	Không tin cậy, có thứ tự	Giao nếu channelSequence mới hơn gói đã giao; gói cũ bị huỷ và đếm StalePacketsDropped	Snapshot server → client
2	Tin cậy, có thứ tự	Cửa sổ 256 ô: gói đúng thứ tự thì giao và xả các gói liền sau đang đệm; gói sớm được chép vào bộ đệm; gói trùng hoặc quá cửa sổ bị bỏ	Sự kiện, spawn, chat
3	Không tin cậy, có thứ tự	Như kênh 1 nhưng bộ đếm riêng	Input client → server

Giải pháp chống head-of-line blocking: độ tin cậy áp dụng **theo kênh**. Thí nghiệm --phase4-report cho thấy khi kênh 2 đang thiếu sequence 0, snapshot kênh 1 vẫn được giao ngay (1 gói), còn kênh 2 đệm gói sau và chỉ giao cả hai khi gói thiếu tới. Nếu dùng TCP, snapshot phải chờ sự kiện bị mất.

4.5. Luồng cắt và ghép mảnh bản tin lớn

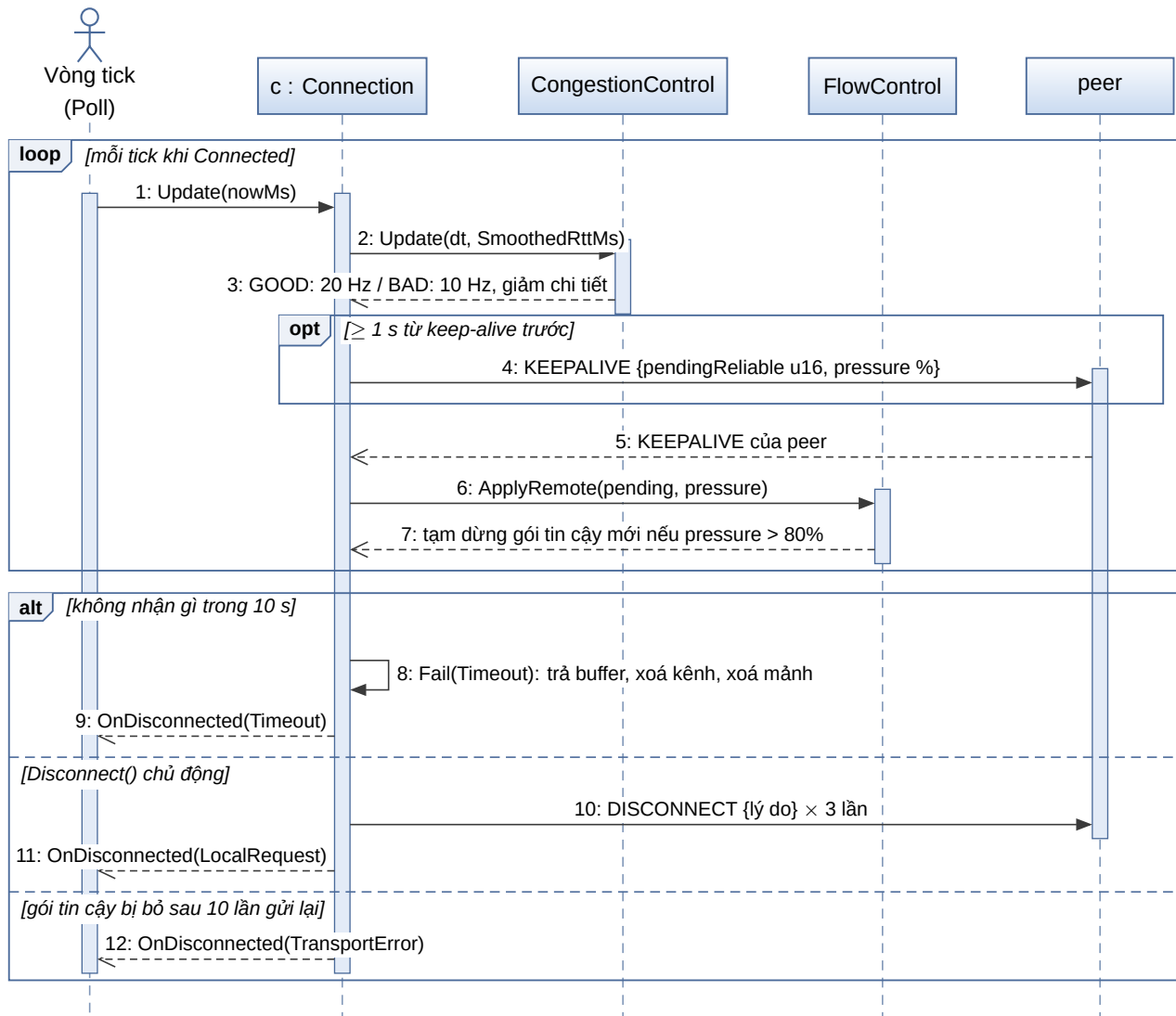
Datagram giới hạn 1200 byte để không bị phân mảnh ở tầng IP trên đường có PPPoE, VPN hay đường hầm (payload tối đa 1184 byte sau header 16 byte). Bản tin lớn hơn — ví dụ danh sách người chơi khi mới vào trận — được cắt ở tầng Transport.



Hình 4.4. Biểu đồ tuần tự cắt và ghép mảnh với bộ nhớ có giới hạn

Giải pháp. (1) Mảnh luôn được gửi **tin cậy** kể cả khi tầng trên yêu cầu không tin cậy, vì thiếu một mảnh thì cả bản tin vô dụng. (2) Giới hạn bộ nhớ trước dữ liệu do kẻ tấn công điều khiển: tối đa 64 mảnh mỗi nhóm, 8 nhóm chờ đang mỗi kết nối, nhóm quá 2 giây bị hủy, và nếu count của một mảnh khác với nhóm thì hủy cả nhóm. (3) Trước khi cắt, kiểm tra số gói tin cậy chưa ack cộng số mảnh không vượt 64, tránh một bản tin lớn chiếm hết cửa sổ. Kiểm thử: ghép đúng từng byte khi mảnh đến ngược thứ tự với bản tin 20 KB.

4.6. Luồng giữ kết nối, kiểm soát nghẽn và ngắt kết nối



Hình 4.5. Biểu đồ tuần tự giữ kết nối, điều khiển luồng và các đường kết thúc kết nối

Vấn đề	Giải pháp đã áp dụng
NAT xoá ánh xạ cổng khi kết nối im lặng; phát hiện client đã mất	KEEPALIVE mỗi 1 s; không nhận gói nào trong 10 s thì coi như rời trận (đúng yêu cầu “mất kết nối quá 10 giây” trong đề)
Keep-alive không bao giờ được gửi trên kết nối đang bận	Hẹn giờ theo lần gửi keep-alive cuối, không theo lần gửi gói bất kỳ; nếu không, kết nối gửi snapshot 20 Hz không bao giờ gửi keep-alive và thông tin điều khiển luồng không được làm mới
Báo ngắt kết nối có thể mất	DISCONNECT gửi 3 lần liên tiếp, không cần ack; bên kia vẫn có lưới an toàn timeout 10 s
Mạng nghẽn làm RTT tăng	Hai chế độ có trễ (hysteresis): $SRTT > 250$ ms chuyển BAD (snapshot 10 Hz, giảm chi tiết); về GOOD khi $SRTT < 200$ ms và đã ở BAD đủ 10 s; nếu vừa về GOOD chưa tới 10 s đã xấu lại thì phạt 20 s để không dao động
Rò rỉ tài nguyên khi đóng	Fail idempotent: trả mọi buffer của ReliabilityLayer, ChannelSet, FragmentAssembler; server trả connectionId về hàng đợi và gỡ ràng buộc playerId để người chơi kết nối lại được

4.7. Bộ giả lập mạng, ghi phát lại gói và đo hiệu năng

Bộ giả lập mạng. NetworkSimulator nằm ngay dưới UdpPeer.Send: mỗi datagram được tung xúc xắc để bỏ, nhân đôi hoặc giữ lại một khoảng trễ (có jitter, có thể bị đẩy chậm thêm để đảo thứ tự), rồi được xử theo thời điểm đến hạn trong Po11. Hạt giống cố định nên cùng cấu hình cho cùng chuỗi sự cố — lỗi tái hiện được. Bật mà không cần build lại: IRONFRONT_SIM=typical.

Hồ sơ	Trễ	Jitter	Mất gói	Đảo thứ tự	Trùng gói
lan	1 ms	0	0	0	0
good	30 ms	5 ms	0,5%	0	0
typical	50 ms	20 ms	5%	2%	0
bad	100 ms	50 ms	15%	5%	2%
awful	150 ms	100 ms	30%	10%	5%

Ghi và phát lại gói. Đặt IRONFRONT_PCAP=đường-dẫn thì PacketLogger ghi mọi datagram vào tệp .ifpcap có phiên bản (chiều, thời điểm, địa chỉ, byte thô). Công cụ Ironfront.Tools.PacketReplay đọc lại, lọc theo khoảng thời gian và connectionId, giải header, ước lượng lỗi hỏng sequence và ghép ack với RTT (Hình 3.10). Vì tệp có thể chứa joinTicket, tệp chỉ lưu trong thư mục bị git bỏ qua.

Kết quả đo (localhost, .NET 8 Release, ghi trong plans/transport/reports/measurements.csv).

Phép đo	Kết quả
Phân tích header GSP	3,5 ns/lần, 0 byte cấp phát
16 kết nối, 30 Hz, 10 phút	16/16 kết nối ổn định, bộ đệm không phải cấp thêm
64 kết nối, 30 Hz, 20 s (đã khởi động)	64/64 kết nối; CPU 0,39% một nhân; Send và Po11 cấp phát 0 byte
Ack bitfield bật / tắt	Một gói ack xác nhận 33 gói / 1 gói; còn 7 / 39 gói tin cậy chờ
Kiểm soát nghẽn	RTT cao: 20 → 10 Hz; thời gian ở BAD tối thiểu 20 s khi vừa chuyển trạng thái

4.8. Kiểm thử phân hệ

Bộ Ironfront.Net.Transport.Tests gồm 85 test xUnit, chạy trong CI trên Ubuntu và Windows:

Tập test	Số test	Kịch bản tiêu biểu
ControlAndSocketTests	22	Gói điều khiển, timeout 10 s, DISCONNECT, socket thất trên localhost
ReliabilityLayerTests	19	Ack bitfield, sequence quay vòng, Karn, RTO, bỏ gói sau 10 lần
HandshakeHardeningTests	12	Cookie sai, cookie hết epoch, vé sai, vé trùng playerId, lứt yêu cầu
ChannelAndFragmentTests	11	Sai thứ tự, gói trùng, gói cũ trên kênh 1/3, ghép mảnh ngược thứ tự, giới hạn 8 nhóm
ReliableChannelSurvivalTests	8	Kênh tin cậy sống sót qua mất gói kéo dài của bộ giả lập
UdpPeerTests	8	Ngân sách 1024 gói/Poll, bộ đệm socket, gói header sai
PacketLoggerTests, Diagnostics	5	Ghi – đọc lại .ifpcap, định dạng overlay chẩn đoán